

SD

A decision framework for comparing HR AI platforms, agentic architectures, and workforce technology options

How to compare without reducing the discussion to feature checklists

Declared vs demonstrated

Fit-to-purpose, not highest spec

Configure vs compose

Discriminating scorecard

Product vs platform

Scaling frontier

Conversation vs autonomy

Interoperability

Siddhartha Dhamankar

Global HR IT Innovation Lead
2026

Personal point of view · Not for distribution

Contents

Preface	Why feature checklists fail	3
P1	Separate declared from demonstrated	4
P2	Separate the product from the technology platform	5
P3	Best is fit-to-purpose, not highest spec	6
P4	Judge against the scaling frontier	7
P5	A product you configure vs a capability you must build	8
P6	Weigh conversation against autonomy	9
P7	Score on dimensions that actually discriminate	10
P8	Design for interoperability, not lock-in	11
Coda	The decision sequence	12

Preface. Why feature checklists fail

Every vendor can produce a checklist where every box is ticked. I have sat through enough of these to know the checklist is where good decisions go to die. The problem is not that features do not matter. It is that a checklist flattens four things that must be kept separate.

What a platform claims	vs what it proved
What a platform is	vs what it runs on
What is good on paper	vs what is good for this specific job
What works today, bounded	vs what must scale, open-ended

This framework is the decision lens I actually use. It is built to force those distinctions into the open, so a comparison produces a defensible decision, not a scorecard.

Principle 1. Separate declared from demonstrated

This is the single most important discipline in any platform evaluation, and it is the one checklists erase. In vendor evaluations, I score every capability not on whether the vendor said they had it, but on whether they showed it under our scenarios.

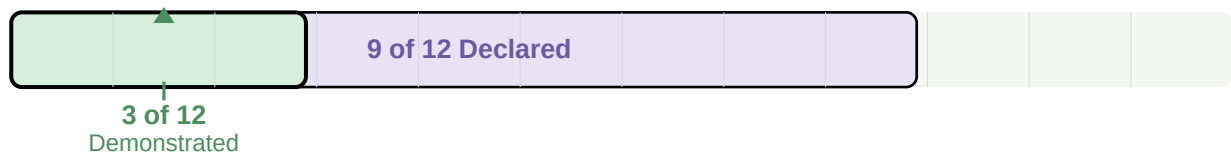


Figure 1. Declared vs demonstrated capability split. A demonstrated 2 beats a declared 3.

The anchor finding of the Moveworks assessment was exactly this split: **9 of 12 capabilities declared, only 3 of 12 demonstrated**. I insisted that the 9/12 vs 3/12 number be treated as the headline for any investment decision, not the glossy capability list.

How to apply it

- D Score declared vs demonstrated separately**
Never blend them into a single number. Declared is an unverified claim carrying delivery risk, not a feature you own.
- ! Watch for capabilities that cannot be demonstrated**
In one ESS evaluation, explainability was simply listed as TBD. Nothing in the recordings or notes showed agent thought-logs or reasoning traces. A gap you cannot even demonstrate is a bigger red flag than a low score.

The rule. A demonstrated 2 beats a declared 3. Always.

Principle 2. Separate the product from the technology platform

One of the most consequential distinctions I draw, and one checklists collapse entirely, is between **the product** and **the technology it runs on**. Conflating them creates real strategic risk.

The product	The technology platform
Has a brand, persona strategy, domain architecture, roadmap, and governance model.	Provides the agent framework, channels, orchestration runtime, and security model.
It is the asset you invest in and own.	If the technology changes, the product should remain.
You are responsible for its roadmap.	You are choosing a technology to build your product on.

Two classic errors this distinction prevents

1 Mistaking the SaaS platform for the product

The view that a third-party platform equals your product is a fundamental misalignment. You are choosing a technology to build your product on, not outsourcing your product.

2 Comparing at the wrong layer

A vendor's chat user experience and a platform's orchestration runtime are not the same kind of thing. Decide which layer you are actually choosing: experience product, implementation platform, or capability provider. Then compare like with like.

Principle 3. Best is fit-to-purpose, not highest spec

The instinct behind a checklist is that more is better and newer is best. That instinct is wrong. I make this argument most directly on model selection: a higher version number does not mean best across the board, because the value case for each model is different.

Purpose	Model choice	Reasoning
Orchestration	GPT 5.0	Good at routing. Very good orchestration already achieved with earlier models.
Response generation	GPT 5.4	Deep reasoning and richer response quality matter here.
Not adopted	GPT 5.5	Significant cost profile. Value case did not justify it for the job at hand.

The generalisable rule. Map each option to the specific job it will do, then judge it on that job. The best orchestrator and the best response generator are different choices. The right question is fit to the actual and future job, not raw feature count.

Principle 4. Judge against the scaling frontier

A platform that is perfect for the first use case can be a dead end for the tenth. The comparison question is: does the boundary of this platform contain my whole roadmap, or just my first release?

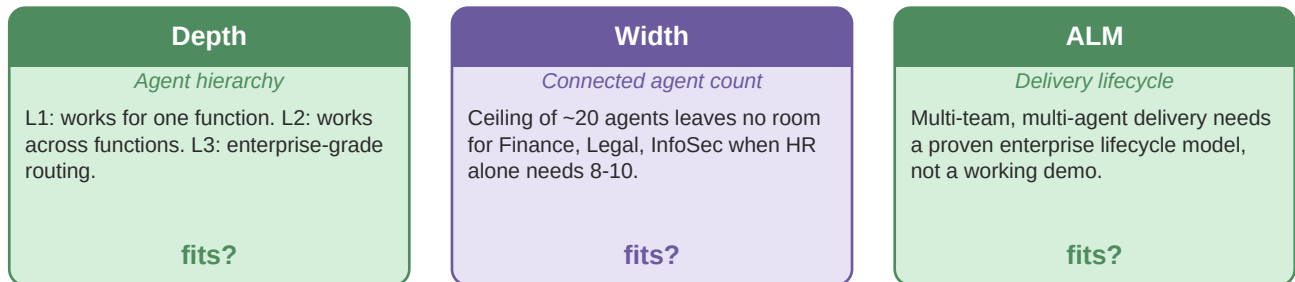


Figure 5. Three scaling dimensions. Evaluate against the roadmap, not the pilot.

A platform may be strongest where the workflow is predefined, integration already exists, the task is bounded, and the answer pattern is controlled. It may be weakest where the enterprise needs open-ended agentic orchestration, deep personalisation, transparent reasoning, large-scale knowledge accuracy, multi-agent interoperability, or emotionally intelligent handling of sensitive HR scenarios.

That is not a knock on any vendor. It is a statement about where the boundary sits.

The rule. Evaluate against the roadmap, not the pilot. A workaround must never quietly become the accepted enterprise target architecture without explicit, recorded risk acceptance.

Principle 5. A product you configure vs a capability you must build

Two options can reach the same functional outcome by very different routes. The route carries cost, risk, and time that a feature checklist never shows. One concrete example: Unity Catalog is a product. The equivalent on another platform is an architecture you build. One is governed on day one with no integration work. The other can achieve equivalent outcomes but through a composed layer carrying engineering cost, integration risk, and ongoing operational complexity.

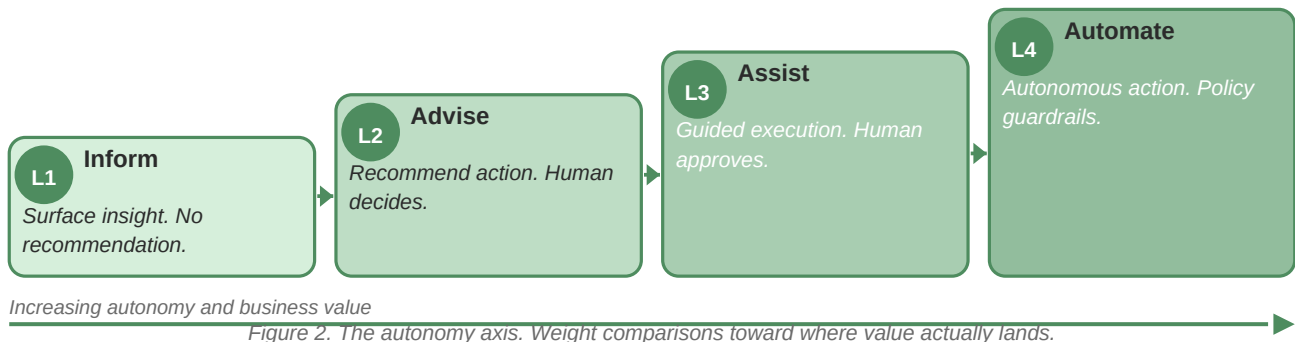
vs	
Configure (buy)	Compose (build)
Faster to governed state	More control and portability
Lower initial build cost	Higher engineering investment
Vendor roadmap dependency	Full architectural ownership
Less flexibility	Ongoing operational complexity
Best when scope is stable	Best when scope is open-ended

Figure 4. Configure vs compose. Neither is wrong. Both must be made with open eyes.

Neither choice is wrong. But both must be made with open eyes. Anchor those decisions on total cost of ownership and go-to-market opportunity, not on which demo looked slicker.

Principle 6. Weigh conversation against autonomy

A checklist treats every capability as equal weight. They are not. Autonomous intelligence is where the business value lies, not conversational AI. A platform that answers beautifully but cannot act is solving the low-value half of the problem.



Use the Inform to Advise to Assist to Automate progression as an evaluation axis. For each platform under comparison, ask: how far along this axis can it take my highest-value workflows? Then weight the comparison accordingly.

Weight the comparison toward the capabilities that move a platform along this axis for your highest-value workflows. Not toward the conversational surface that every vendor demos well.

Principle 7. Score on dimensions that actually discriminate

When a structured scorecard is warranted, build it around the dimensions that separate serious enterprise platforms from bounded tools. In staffing technology evaluations, the discriminating dimensions were not a generic feature list.

Dimension	What it actually tests
Global standard process fit	Does the platform support the actual business process across regions and entities?
Agentic functionality	Can it move along the Inform-Advise-Assist-Automate axis for the real workflow?
Federal and regulatory compliance	Security, data residency, and compliance posture for the operating context.
Technical fit	Scalability, performance, and integration maturity against the actual estate.
Experience	Meeting people where they are. In-app quality across channels.
Delivery agility	Release management cadence and backlog responsiveness.
Innovation trajectory	R+D investment direction and go-to-market potential.
Total cost of ownership	Technology cost, time-to-implement, cost-to-implement, and incremental ops cost.

The score narrows the field. The rationale makes the decision. Always pair every score with a plain-language recommendation and rationale. Name the winner and why. Name the strong alternative and its trade-off. Name the least favourable option and what makes it so.

Principle 8. Design for interoperability, not lock-in

A platform is never the whole estate. Increasingly the right question is not "does this platform do X?" but "does it federate with the platforms that already do X?" Enterprise agent federation, integrating with agents from systems such as Workday rather than replicating their logic, is a core architectural requirement, enabled by open protocols such as Agent-to-Agent (A2A) and Model Context Protocol (MCP).

Prefer	Avoid
Federates with existing specialist agents via open protocols	Demands logic be rebuilt inside its own walls
Supports hand-off and collaboration across platforms	Treats federation as a future roadmap item
Uses standard protocols (A2A, MCP) for agent interoperability	Uses proprietary connectors that create long-term lock-in
Keeps the workforce on a single surface regardless of backend	Requires re-platforming existing workflows to join the ecosystem

Credit the option that plays well with others. An ecosystem of federated specialist agents, each excellent at its domain, will outperform a monolithic platform trying to do everything itself.

Coda. The decision sequence

When I run a real comparison, I move through these questions in order. Each one filters the field before the next.

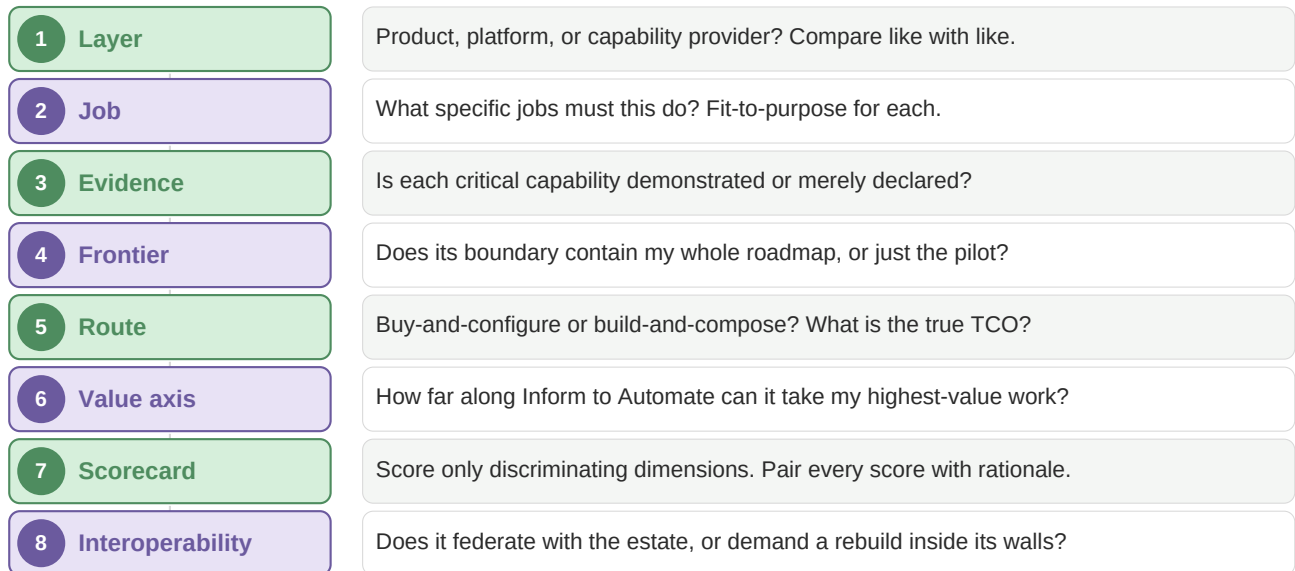


Figure 3. The decision sequence. Eight filters applied in order.

The bottom line. A feature checklist tells you what a platform has. This framework tells you what a platform will do for you, at scale, over time, against a specific job. And, just as importantly, what it will cost you to find out you were wrong. That is the difference between a scorecard and a decision.

Drawn from my own authored assessments and positions: the Moveworks and Copilot Studio evaluations (declared-vs-demonstrated, platform decision lens), the staffing technology capability comparison and CIO point of view, the employee self-service vendor evaluation, the People Assist product-vs-platform distinction and architecture-options analysis, the LLM model-fit recommendation, and the data-governance platform comparison (2024-2026). Views expressed are personal.